SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Malli Chandana	**Reg. No.:**	192472250
Date:	03-08-2026	**Duration:**	60 minutes

Code of Conduct
I Malli Chandana(192472250)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: Malli Chandana

Annexure A – Architecture Design Assignment

Software Requirement Specification (SRS) for Eco-Tourism Management and Sustainable Travel Recommendation System

1 – SYSTEM REQUIREMENT ANALYSIS

1.1 System Objectives

The **Eco-Tourism Management and Sustainable Travel Recommendation System** is designed to promote sustainable tourism by providing travelers with a digital platform to discover eco-friendly destinations, accommodations, and travel services. The system aims to simplify trip planning by offering AI-based travel recommendations, secure online booking, and reliable information about sustainable tourism. It also supports tourism authorities by improving destination management, monitoring tourist activities, and encouraging environmental conservation. Overall, the system enhances the travel experience while promoting responsible tourism and supporting local communities.

Key Points

- Promote sustainable tourism.
- Provide AI-based travel recommendations.
- Simplify travel planning.
- Support eco-friendly destinations.
- Improve destination management.

- Encourage environmental conservation.
- Enhance user experience.

1.2 Functional Requirements

Functional requirements describe the core features that the system must provide. Users should be able to register, log in, search eco-tourism destinations, receive personalized recommendations, book accommodations, make secure payments, and submit reviews. Administrators should manage users, destinations, bookings, and reports. These functions ensure smooth operation and improve the overall efficiency of the tourism platform.

Key Points

- User registration and login.
- Destination search.
- AI travel recommendations.
- Online booking.
- Secure payment.
- Ratings and reviews.
- Admin management.

1.3 Non-Functional Requirements

Non-functional requirements define the quality and performance of the system. The application should provide fast response times, secure user authentication, reliable performance, and an easy-to-use interface. It should also be scalable to support more users in the future and maintainable for regular updates and improvements.

Key Points

- High performance.
- Secure authentication.
- Reliable operation.
- User-friendly interface.
- Scalable system.
- Easy maintenance.
- High availability.

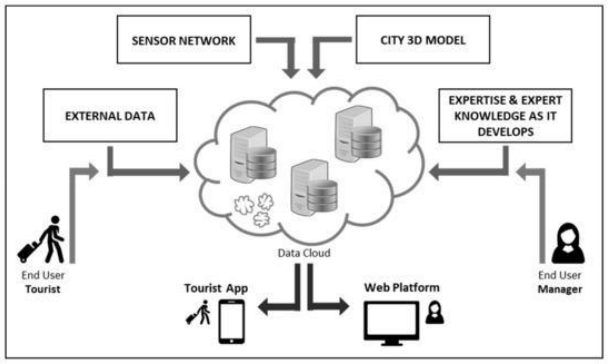
1.4 Quality Attributes

Quality attributes ensure that the software performs efficiently and meets user expectations. The proposed architecture should support **scalability** to handle increasing users, **maintainability** for easy updates, **reliability** for continuous operation, **availability** with minimum downtime, **performance** through quick response times, and **security** to protect user data and online transactions. These attributes contribute to a robust and future-ready software system.

Key Points

- **Scalability:** Supports growing users.

- **Maintainability:** Easy updates.
- **Reliability:** Stable operation.
- **Availability:** 24×7 access.
- **Performance:** Fast response.
- **Security:** Protects user data.
- **Future-ready architecture.**



2 – SELECT A SUITABLE SOFTWARE ARCHITECTURE

2.1 Selected Software Architecture

The **Hybrid Architecture (MVC + Client–Server)** is selected for the **Eco-Tourism Management and Sustainable Travel Recommendation System** because it provides a well-structured, scalable, and secure solution. The **Model-View-Controller (MVC)** architecture separates the user interface, business logic, and data management into independent components, making the application easier to develop, test, and maintain. The **Client–Server** architecture allows users to access the platform through web or mobile devices while the server processes requests, manages business logic, and stores data in a centralized database. This combination provides better performance, flexibility, and maintainability, making it suitable for an online tourism management system.

Key Points

- Hybrid Architecture (MVC + Client–Server).
- Separates presentation and business logic.
- Centralized data management.
- Supports web and mobile users.
- Easy to maintain and upgrade.
- Secure communication.
- Suitable for large-scale applications.

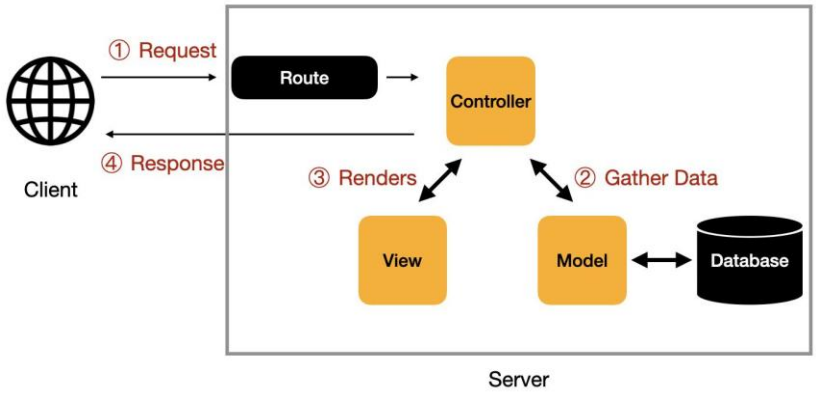
2.2 Justification for the Selected Architecture

The Hybrid Architecture is the most suitable choice because the proposed system includes multiple modules such as user management, destination search, AI-based recommendations, online booking, payment processing, and report generation. MVC improves code organization by separating different functionalities, while Client–Server enables centralized data storage and efficient communication between users and the application. This architecture also supports future integration with external APIs, cloud services, AI modules, and payment gateways. Since tourism platforms require continuous updates and

increasing user traffic, the selected architecture provides high scalability, reliability, and maintainability while ensuring secure data exchange.

Key Points

- Supports modular development.
- Handles multiple system modules.
- Improves scalability.
- Easy system maintenance.
- Secure data communication.
- Supports future integration.
- Enhances overall performance.



3 – SOFTWARE ARCHITECTURE DIAGRAM

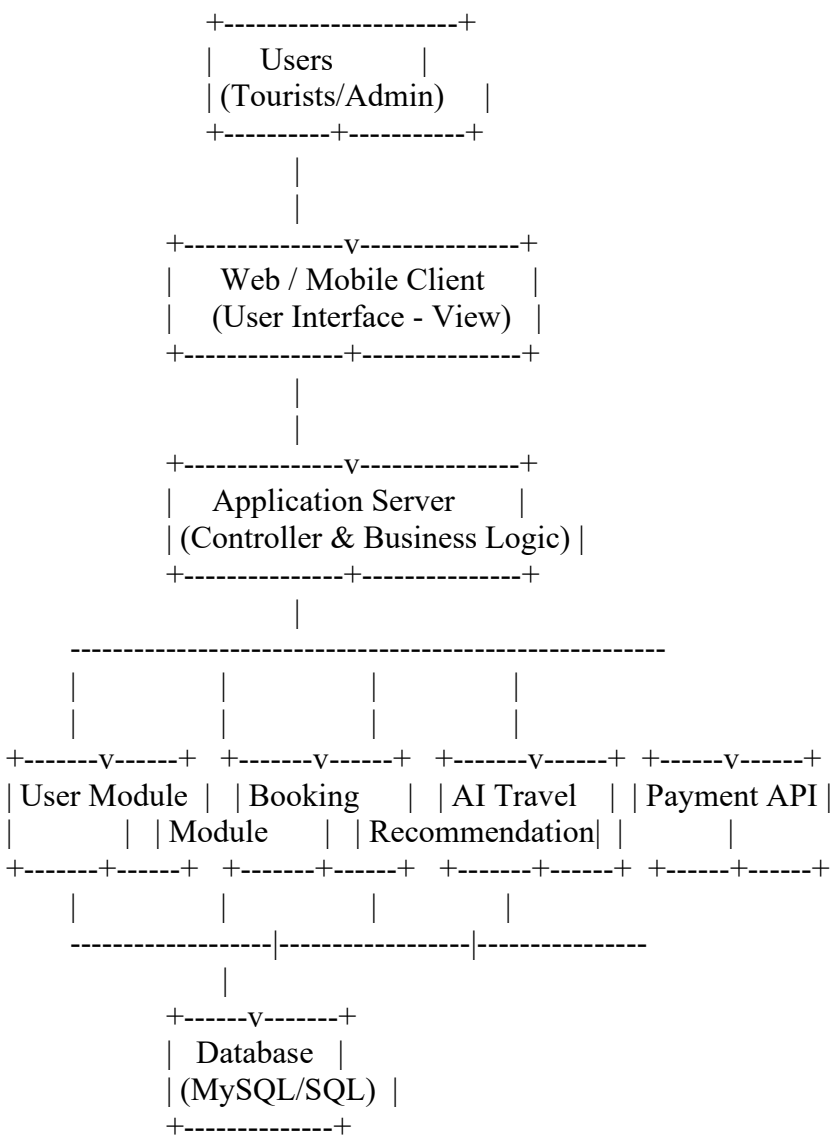
3.1 Architecture Diagram

The proposed **Eco-Tourism Management and Sustainable Travel Recommendation System** follows a **Hybrid (MVC + Client–Server)** architecture. Users access the system through a web browser or mobile application. Their requests are sent to the application server, where the business logic processes user actions such as registration, destination search, travel recommendations, booking, and payments. The server communicates with the database to retrieve or store information and also interacts with external services such as payment gateways, GPS, and AI recommendation APIs. This architecture provides secure communication, centralized data management, and efficient system performance.

Key Points

- Web and mobile client.
- Application server.
- MVC architecture.
- Centralized database.
- AI recommendation API.
- Payment gateway integration.
- GPS/location services.

3.2 Software Architecture Diagram

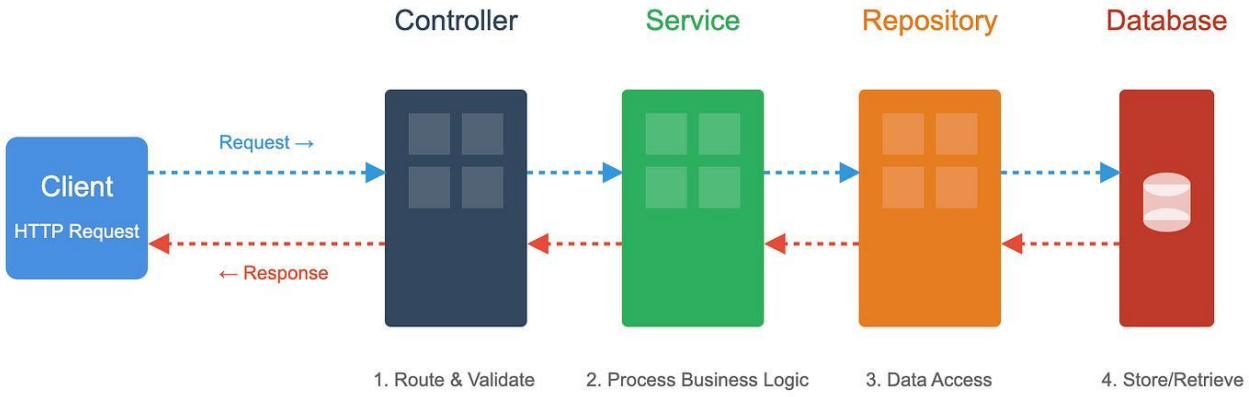


3.3 Communication Mechanism

The client sends requests to the application server using **HTTP/HTTPS**. The server processes the request, interacts with the required modules, retrieves or stores information in the database, and returns the response to the user. External APIs such as payment gateways, GPS, and AI recommendation services communicate securely with the server to provide additional functionality.

Key Points

- Client–Server communication.
- HTTP/HTTPS protocol.
- Database connectivity.
- Secure API integration.
- Real-time data exchange.
- Fast response.
- Reliable communication.



4 – EXPLAIN COMPONENTS AND THEIR INTERACTIONS

4.1 Components and Their Responsibilities

The proposed architecture consists of several components that work together to provide a complete eco-tourism management solution. The **User Interface (View)** allows tourists and administrators to interact with the system through web or mobile applications. The **Application Server (Controller)** processes user requests and manages the business logic. The **User Management Module** handles registration, login, and profile management, while the **Recommendation Module** provides AI-based travel suggestions. The **Booking Module** manages reservations and payments, and the **Database** securely stores all user, destination, and booking information. External services such as payment gateways and GPS APIs support secure transactions and location-based recommendations.

Key Points

- User Interface (Web/Mobile).
- Application Server.
- User Management Module.
- AI Recommendation Module.
- Booking & Payment Module.
- Database.
- External APIs.

4.2 Component Interactions

All components communicate through the application server. When a user searches for an eco-tourism destination, the request is sent from the user interface to the server. The server processes the request, retrieves data from the database, and if needed, communicates with the AI recommendation service and GPS API. The processed information is then returned to the user. Similarly, booking requests are validated, payment is processed through the payment gateway, and booking details are stored in the database. This interaction ensures smooth communication and reliable system performance.

Key Points

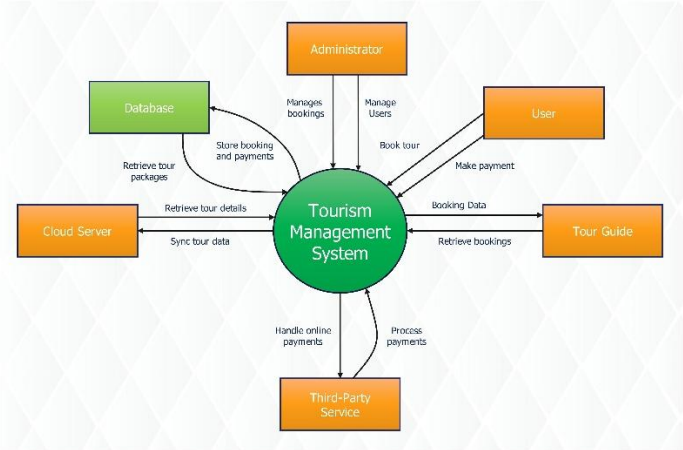
- User sends request.
- Server processes request.
- Database stores and retrieves data.
- AI generates recommendations.
- Payment gateway processes payments.
- Response sent to user.
- Secure communication.

4.3 System Workflow

The workflow begins when the user logs into the system and searches for eco-tourism destinations. The application server validates the request and retrieves destination information from the database. The AI module generates personalized recommendations based on user preferences. The user can then select a destination, complete the booking process, make a secure payment, and receive confirmation. Finally, users can provide ratings and reviews, while administrators monitor bookings and manage system data through the admin dashboard.

Key Points

- User login.
- Destination search.
- AI recommendation.
- Booking confirmation.
- Secure payment.
- Feedback submission.
- Admin monitoring.



---

5 – DISCUSS QUALITY ATTRIBUTES

5.1 Quality Attribute Analysis

The proposed **Hybrid (MVC + Client–Server)** architecture is designed to achieve high software quality by satisfying important quality attributes. It provides a scalable and maintainable structure that supports increasing users and future system enhancements. The architecture also ensures secure communication,

reliable operation, fast performance, and high availability through proper separation of components and centralized data management. These quality attributes improve user satisfaction and ensure the long-term success of the Eco-Tourism Management and Sustainable Travel Recommendation System.

Key Points

- Supports high software quality.
- Improves user experience.
- Ensures reliable performance.
- Provides secure communication.
- Easy to maintain.
- Supports future expansion.
- Suitable for large-scale systems.

5.2 Quality Attributes

Scalability

The architecture can support a growing number of users, destinations, and bookings without affecting system performance. Additional servers or cloud resources can be added whenever demand increases.

Security

User data and online transactions are protected using secure authentication, encrypted passwords, HTTPS communication, and trusted payment gateways.

Performance

The system provides fast response times through optimized database queries, efficient business logic, and centralized processing.

Reliability

Regular backups, error handling, and database recovery mechanisms ensure continuous and stable system operation with minimal failures.

Maintainability

The MVC architecture separates the user interface, business logic, and database, making the software easier to update, debug, and maintain.

Availability

The system is designed to provide **24×7 availability**, allowing users to access tourism services anytime through web or mobile applications.

Key Points

- **Scalability:** Supports more users.

- **Security:** Protects user data.
- **Performance:** Fast response time.
- **Reliability:** Stable operation.
- **Maintainability:** Easy updates.
- **Availability:** 24×7 service.
- **Supports future growth.**

---

6 – JUSTIFY ARCHITECTURAL DECISIONS

6.1 Architectural Decisions

The **Hybrid (MVC + Client–Server)** architecture was selected because it provides a structured, scalable, and secure solution for the **Eco-Tourism Management and Sustainable Travel Recommendation System**. MVC separates the application into the **Model, View, and Controller**, making the system easier to develop, test, and maintain. The Client–Server architecture enables centralized data management and allows users to access the system through web and mobile devices. This combination supports efficient communication, better performance, and future system expansion.

Key Points

- Hybrid architecture selected.
- MVC separates application layers.
- Client–Server enables centralized management.
- Supports web and mobile access.
- Easy development and maintenance.
- Secure communication.
- Future-ready design.

6.2 Trade-Offs Considered

While the Hybrid architecture provides many advantages, it also involves certain trade-offs. Developing multiple layers and integrating external APIs increases the initial development effort and cost. The system also depends on a stable internet connection and proper server management. However, these challenges are outweighed by benefits such as better scalability, improved security, easier maintenance, and support for future enhancements. Therefore, the selected architecture provides a balanced solution for the proposed system.

Key Points

- Higher initial development cost.
- Requires internet connectivity.
- More complex implementation.
- Better scalability.
- Improved maintainability.
- Enhanced security.
- Long-term benefits.

6.3 Future Enhancements and Long-Term Maintainability

The proposed architecture is flexible and supports future improvements without major changes to the existing system. New features such as **Augmented Reality (AR)**, **IoT-based environmental monitoring**, **advanced AI recommendations**, **multilingual support**, and **cloud services** can be integrated easily. The modular design also allows developers to update or replace individual components without affecting the entire application. This ensures long-term maintainability and supports the future growth of the tourism platform.

Key Points

- Easy feature integration.
- Supports AR and IoT.
- Advanced AI modules.
- Cloud integration.
- Modular architecture.
- Long-term maintainability.
- Future scalability.

Conclusion

The **Hybrid (MVC + Client–Server)** architecture is the most suitable choice for the **Eco-Tourism Management and Sustainable Travel Recommendation System** because it provides a secure, scalable, and maintainable structure. The architecture efficiently supports user management, AI-based travel recommendations, online booking, and centralized data storage while ensuring high performance and reliability. Its modular design makes future enhancements and system integration easier, allowing the platform to adapt to changing technologies and user requirements. Overall, the selected architecture provides a strong foundation for developing a reliable and sustainable eco-tourism management system.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

	organized, professional report.			

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25